

# ANALYSTLAB AFRICA EXPERIENCE LAB

HealthConnect Clinic — Week 5

---

## ML Pipeline Development & Initial Implementation Report

---

**Intern Name:** Grai Rudolf  
**Track:** Machine Learning Engineering  
**Role:** Junior Machine Learning Engineer  
**Date:** September 2026

**Project:** HealthConnect Clinic  
**Document Version:** 1.0  
**Stage:** Initial Implementation  
**Status:** Baseline Deliverable

### Abstract

This document presents the Week 5 practical implementation of the HealthConnect Experience Lab from the Machine Learning Engineering track. Building directly on the Week 4 problem definition and production system design, we deliver a modular, reusable ML pipeline comprising data cleaning with explicit leakage control, a reusable preprocessing transformer, behavioural feature engineering, and an initial model workflow that trains and benchmarks five baseline classifiers. The primary baseline (Logistic Regression) achieves a held-out recall of 0.65 and ROC-AUC of 0.66, confirming that pre-appointment signals carry meaningful predictive value. We also document component testing evidence, implementation issues, dependencies, and a clear focus for Week 6.

## Contents

|           |                                                           |          |
|-----------|-----------------------------------------------------------|----------|
| <b>1</b>  | <b>Part 1 — Review of Week 4 Foundation</b>               | <b>2</b> |
| 1.1       | Problem Defined in Week 4 . . . . .                       | 2        |
| 1.2       | Week 4 Proposed Approach . . . . .                        | 2        |
| 1.3       | Relevant HealthConnect Resources . . . . .                | 2        |
| 1.4       | Key Assumptions & Limitations from Week 4 . . . . .       | 2        |
| 1.5       | Proposed Week 5 Focus (as planned in Week 4) . . . . .    | 2        |
| <b>2</b>  | <b>Repository Structure &amp; Pipeline Architecture</b>   | <b>2</b> |
| <b>3</b>  | <b>Data Processing Workflow</b>                           | <b>3</b> |
| 3.1       | Dataset Inspection . . . . .                              | 3        |
| 3.2       | Cleaning & Binary Target Construction . . . . .           | 3        |
| 3.3       | Reusable Preprocessing Transformer . . . . .              | 3        |
| <b>4</b>  | <b>Feature Engineering</b>                                | <b>3</b> |
| <b>5</b>  | <b>Train / Test Strategy</b>                              | <b>4</b> |
| <b>6</b>  | <b>Baseline Model Development</b>                         | <b>4</b> |
| <b>7</b>  | <b>Initial Evaluation</b>                                 | <b>4</b> |
| 7.1       | Primary Baseline — Logistic Regression . . . . .          | 4        |
| 7.2       | Interpretation . . . . .                                  | 6        |
| 7.3       | Feature Importance . . . . .                              | 6        |
| <b>8</b>  | <b>Component Testing Evidence</b>                         | <b>6</b> |
| <b>9</b>  | <b>Assumptions, Limitations, Risks &amp; Dependencies</b> | <b>7</b> |
| 9.1       | Review of Week 4 Items . . . . .                          | 7        |
| 9.2       | Implementation Issues . . . . .                           | 7        |
| 9.3       | Dependencies . . . . .                                    | 7        |
| <b>10</b> | <b>Cross-Track Collaboration</b>                          | <b>7</b> |
| <b>11</b> | <b>Recommended Focus for Week 6</b>                       | <b>8</b> |

## 1 Part 1 — Review of Week 4 Foundation

### 1.1 Problem Defined in Week 4

HealthConnect Clinic experiences high patient no-shows. In Week 4, the Machine Learning track framed the task as a supervised binary classification problem,

$$P(\text{No-Show}_i \mid X_i) = \sigma(f(X_i)), \quad (1)$$

where  $Y_i = 1$  denotes a No-Show and  $Y_i = 0$  denotes Attended, using only pre-appointment features available  $T - 48$  hours before the scheduled appointment.

### 1.2 Week 4 Proposed Approach

A 5-stage production architecture was proposed: EHR Ingestion → Feature Store & Schema Validator → ML Inference Microservice → Actionable Decision Engine → MLOps & Drift Monitoring. Patients are stratified into High, Moderate and Low risk tiers to trigger automated multi-channel engagement.

### 1.3 Relevant HealthConnect Resources

The implementation relies on `HealthConnect_Appointment_Data.csv` (5,000 records across 1,696 patients), `HealthConnect_Data_Dictionary.xlsx`, and the `HealthConnect_Clinic_Knowledge_Base.d`.

### 1.4 Key Assumptions & Limitations from Week 4

- Cancelled appointments (5.26%) are excluded from binary modelling.
- `waiting_time_minutes` is a post-arrival metric and is excluded to prevent target leakage.
- Cross-validation should be grouped by `patient_id` or temporal to avoid repeat-patient leakage.
- `reminder_channel` missingness is NMAR (missing means no reminder sent).

### 1.5 Proposed Week 5 Focus (as planned in Week 4)

Build modular preprocessing pipelines, engineer behavioural features, and train/benchmark baseline classifiers on recall-focused metrics. This report delivers exactly that scope.

## 2 Repository Structure & Pipeline Architecture

The Week 5 implementation is organised as a modular Python package:

```
week5/
  AnalystLab_Africa_Week5_Experience_Lab_Assignment.pdf
  README.md
  requirements.txt
  week5_healthconnect_ml_pipeline.ipynb
  pipeline/
    __init__.py
    config.py           # Paths, column lists, hyperparameters
    data_processing.py  # load, clean, preprocessing transformer
```

```
feature_engineering.py
model_training.py    # baseline model definitions & CV
evaluation.py        # metrics & visualisations
utils.py             # seeds, helpers
figures/             # Generated diagnostic figures
tests/
  test_pipeline.py   # Component smoke tests
reports/             # This report & project summary (LaTeX/PDF)
```

The pipeline is deliberately decomposed into single-responsibility modules so that each stage can be reused, tested, and later containerised independently — consistent with the Week 4 microservice architecture.

---

## 3 Data Processing Workflow

### 3.1 Dataset Inspection

The raw dataset contains 5,000 records across 1,696 unique patients. The outcome distribution is No-Show (2,423; 48.46%), Attended (2,314; 46.28%), Cancelled (263; 5.26%).

### 3.2 Cleaning & Binary Target Construction

Consistent with Week 4, the cleaning step:

1. Keeps only **Attended** and **No-Show** records, yielding 4,737 rows.
2. Encodes the target as 1 = No-Show, 0 = Attended.
3. Drops leakage and identifier columns: `appointment_id`, `patient_id`, `waiting_time_minutes`, `booking_date`, `appointment_date`.
4. Imputes `reminder_channel` (NMAR) to an explicit `"None_Sent"` category.
5. Imputes `distance_to_clinic_km` (MCAR) with the median.

### 3.3 Reusable Preprocessing Transformer

A `ColumnTransformer` pipeline applies median imputation + `RobustScaler` to numerical columns and most-frequent imputation + One-Hot encoding to categorical columns. This transformer is a reusable production component that can be re-fit on training data and applied to unseen data.

---

## 4 Feature Engineering

Four behavioural features were engineered, each with explicit justification:

---

Table 1: Engineered Features and Expected Modelling Value

| Feature                 | What Was Created                                                            | Why / Expected Value                        |
|-------------------------|-----------------------------------------------------------------------------|---------------------------------------------|
| historical_noshow_ratio | $\frac{\text{previous\_no\_shows}}{\max(\text{previous\_appointments}, 1)}$ | Captures recidivism; strong no-show signal  |
| lead_time_bin           | Binned booking lead days                                                    | Longer lead times → higher no-show (Week 4) |
| is_new_patient          | 1 if no prior appointments                                                  | Cold-start handling for linear models       |
| has_reminder            | Binary reminder flag                                                        | Communication-channel signal                |

Table 2: Train / Test Split Summary

| Split    | Records | No-Show Rate |
|----------|---------|--------------|
| Training | 3,789   | 0.51         |
| Test     | 948     | 0.51         |

## 5 Train / Test Strategy

A stratified random 80/20 split preserves the no-show proportion in both training and test sets.

**Limitation acknowledged:** For this initial baseline a stratified split is used to establish performance quickly. Because a single patient may contribute multiple appointments, a `GroupKFold` grouped by `patient_id` (and a temporal split) remains the rigorous strategy and is flagged as a Week 6 refinement to fully eliminate repeat-patient leakage.

## 6 Baseline Model Development

Five baseline classifiers were selected for distinct reasons: (a) Logistic Regression as an interpretable linear reference; (b) Decision Tree as an auditable non-linear baseline; (c) Random Forest for robust bagging; (d) Gradient Boosting as a strong boosting baseline; and (e) LightGBM as a candidate for production. All use `class_weight='balanced'` and are compared via 5-fold stratified cross-validated recall.

Table 3: Baseline Model Cross-Validation Recall

| Model               | CV Recall (mean) | CV Recall (std) |
|---------------------|------------------|-----------------|
| Logistic Regression | 0.628            | 0.026           |
| Random Forest       | 0.615            | 0.025           |
| Gradient Boosting   | 0.596            | 0.025           |
| LightGBM            | 0.592            | 0.019           |
| Decision Tree       | 0.558            | 0.040           |

## 7 Initial Evaluation

### 7.1 Primary Baseline — Logistic Regression

Logistic Regression is selected as the primary reporting baseline for this initial stage because it is the most interpretable model.

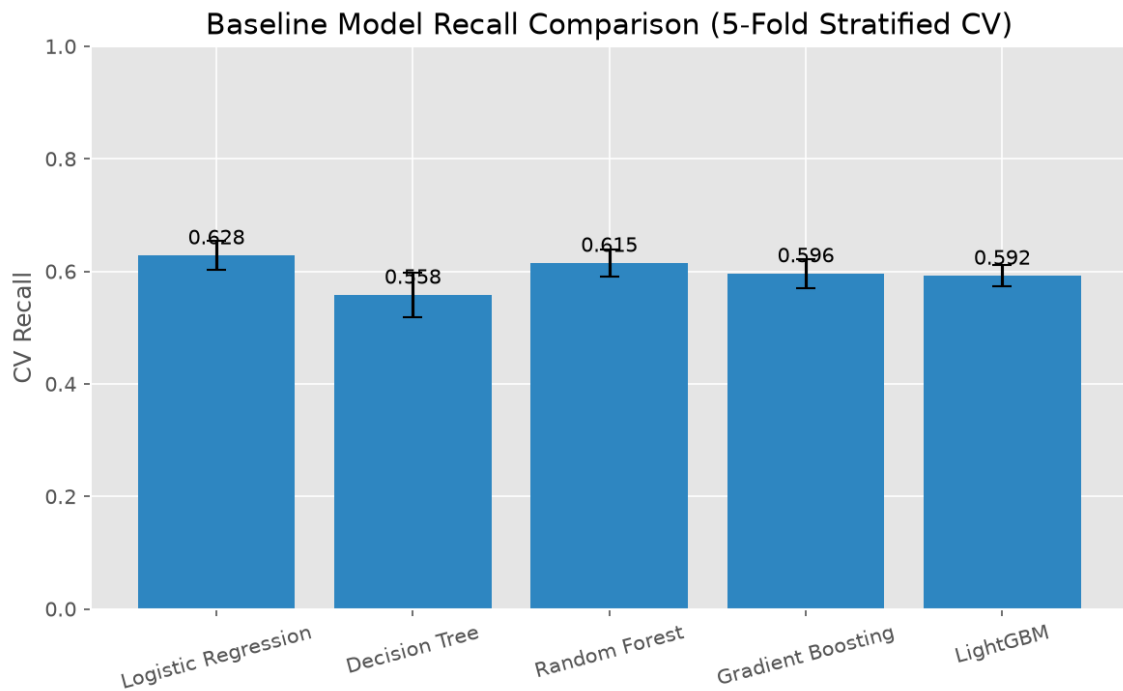


Figure 1: Baseline model recall comparison (5-fold stratified cross-validation).

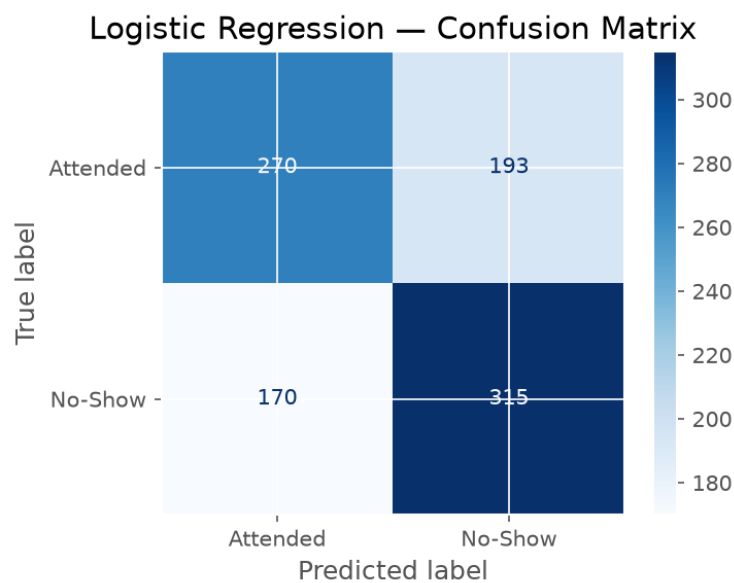


Figure 2: Confusion matrix for the Logistic Regression baseline on the held-out test set.

Table 4: Baseline Model Evaluation on Held-Out Test Set ( $N = 948$ )

| Model               | Acc   | Prec  | Recall | F1    | ROC-AUC |
|---------------------|-------|-------|--------|-------|---------|
| Logistic Regression | 0.617 | 0.620 | 0.650  | 0.634 | 0.664   |
| Random Forest       | 0.617 | 0.623 | 0.639  | 0.631 | 0.652   |
| Decision Tree       | 0.571 | 0.587 | 0.540  | 0.563 | 0.619   |
| Gradient Boosting   | 0.588 | 0.593 | 0.621  | 0.606 | 0.613   |
| LightGBM            | 0.593 | 0.601 | 0.606  | 0.604 | 0.622   |

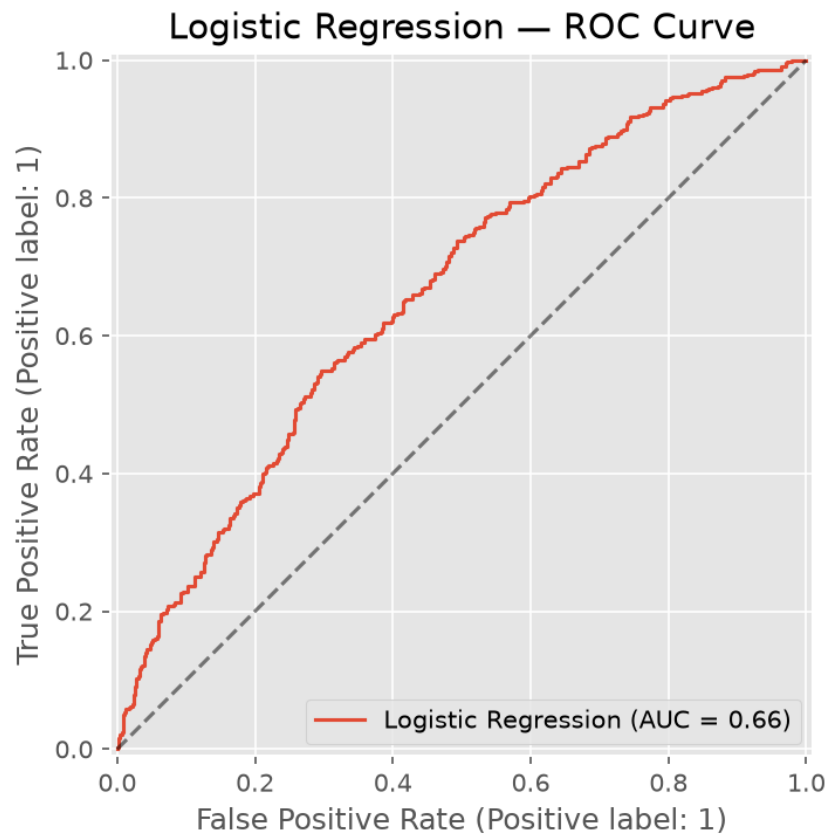


Figure 3: ROC curve for the Logistic Regression baseline (AUC = 0.664).

## 7.2 Interpretation

The primary baseline achieves **accuracy** 0.617, **recall** 0.650, and **ROC-AUC** 0.664. The balanced weighting trades precision for higher no-show recall, the desired business direction: unflagged high-risk patients (false negatives) are costlier than unnecessary reminders. Performance sits well above the 0.5 random baseline, confirming that pre-appointment features carry real predictive signal. This is a reliable baseline, not yet a production-ready system.

## 7.3 Feature Importance

Random Forest feature importances highlight the dominant signals:

`booking_lead_days` dominates, matching the Week 4 finding that longer lead times strongly predict no-shows, followed by `distance_to_clinic_km`, `age`, and prior appointment history — consistent with the planned system design.

# 8 Component Testing Evidence

Basic smoke tests confirm each pipeline component behaves correctly:

- Target is strictly binary; leakage columns are dropped.
- All engineered features are present; no remaining nulls.
- The preprocessing transformer is shape-consistent between fit and transform.
- All baseline models train and predict without error.

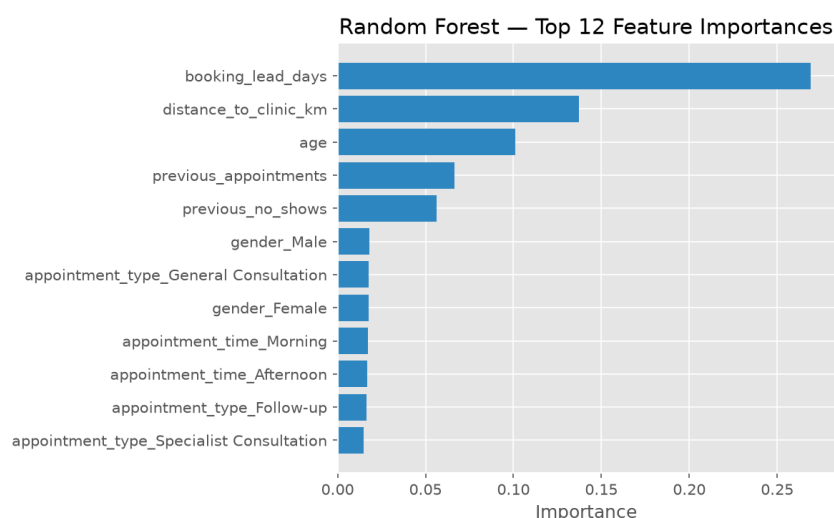


Figure 4: Top Random Forest feature importances.

All 6 tests pass (6/6). See `tests/test_pipeline.py`.

## 9 Assumptions, Limitations, Risks & Dependencies

### 9.1 Review of Week 4 Items

- **Remain relevant:** leakage exclusion, cancellation handling, NMAR imputation, and the grouping requirement for rigorous cross-validation.
- **Changed:** the initial baseline uses a stratified split rather than full `GroupKFold`; this is an acknowledged simplification to be corrected in Week 6.
- **New issue discovered:** the `booking_lead_days = 0` edge case required `include_lowest=True` in binning to avoid NaN labels.

### 9.2 Implementation Issues

- Dropping `Cancelled` records reduces the dataset from 5,000 to 4,737 rows.
- `reminder_channel` required explicit `"None_Sent"` categorisation.
- Repeat-patient leakage control requires `GroupKFold`; deferred to Week 6.

### 9.3 Dependencies

Python 3, Pandas, NumPy, Matplotlib, Scikit-learn, LightGBM (optional), Jupyter; see `requirements.txt`.

## 10 Cross-Track Collaboration

- **Track interacted with:** Data Science / Data Analytics.
- **Information exchanged:** Modelling must use only pre-appointment features; definitions of cancellations, no-shows and leakage boundaries were shared.

- **Why relevant:** Ensures consistency of the target definition and feature scope across tracks so the engineered features and pipeline align with the analytics and data science outputs.
  - **Outcome:** Confirmed the feature set and leakage boundaries used here are aligned across the project team.
- 

## 11 Recommended Focus for Week 6

1. Apply `GroupKFold` grouped by `patient_id` and a temporal split for rigorous leakage control.
2. Tune hyperparameters and decision threshold for a recall-driven objective ( $F_2$  score).
3. Wrap the best model in the containerised FastAPI inference service from the Week 4 architecture.
4. Add MLflow tracking and Evidently AI drift monitoring.